blueAlly

Prepared for: Nitin

Original Test: May 19, 2026

Table of Contents

1 Executive Summary

BlueAlly conducted a penetration test of the Nitin web application and its APIs to identify security weaknesses. The goal of this assessment is aimed at minimizing risks to data, systems, and company reputation. The duration of the pen test ran from 19th May 2026 to 23rd May 2026.

The BlueAlly team assessed the application's functionality, security controls, and potential vulnerabilities. Common weaknesses listed in the OWASP Top 10 and security best practices outlined in the OWASP ASVS were considered. While the pen test consisted of a mixture of automated tools, manual analysis formed the basis of the assessment.

Vulnerabilities identified were categorized as follows:

- Critical: 1
- High: 0
- Medium: 1
- Low: 0
- Informational: 0

2 INTRODUCTION

Background

As part of an ongoing security program, Nitin identified the need to conduct an application security assessment and penetration test of its web application.

This report includes the following:

- BlueAlly's approach to the assessment
- Assessment scope
- Key findings are listed with their qualitative risk assessment
- Detailed recommendations for each finding
- A remediation plan

3 APPROACH

BlueAlly performed a Runtime Web Application Vulnerability Assessment of the Nitin web application components that were defined in the scope.

Web Application Assessment & Penetration Test

- Identify the possible security vulnerabilities and test the possibility of exploitation in the Web application infrastructure
- Assess the security vulnerabilities present in the Web application infrastructure concerning industry standard methodologies like OSSTMM (Open-Source Security Testing Methodology Manual) and OWASP (Open-Web Application Security Project) Web application testing methodology
- Suggest best practices for remediation

Runtime Application Vulnerability Assessment

The Runtime Application Vulnerability Assessment involves detecting security vulnerabilities through detailed examination and testing of the application in a runtime environment. This assessment simulates an attack by a skilled adversary in a controlled setting, allowing the tester to identify the types of vulnerabilities that can be realistically exploited. A Runtime Application Vulnerability Assessment includes the following phases:

- **Information Gathering** — Gain a better understanding of the application by browsing the website as an anonymous visitor, a typical user, and then as a privileged user (where applicable)
- **Attacking Authorization** — Attempt to gain access to data belonging to other users of the system, resources, or functionalities belonging to privileged users
- **Session Management** — Attempt to exploit session management vulnerabilities and manipulate client state
- **Input Validation Attacks** — Attempt to inject invalid, malformed, or unexpected input to trigger unexpected behavior or exceptions in the application or related libraries. This is also done to traverse seldom-used business logic, find unaccounted-for use cases, or bypass application controls. In addition, input-validation attacks attempt to inject instructions like code, commands, and packets into user-controlled input to exploit logic or code that directly interprets or executes user-supplied input.

This assessment report contains:

- Technical details of the vulnerabilities discovered, with substantiation of the exploits
- The risk mitigation recommendations that need to be implemented to ensure that the systems are secure from the risks stemming from the discovered vulnerabilities

4 SCOPE

The assessment was conducted between 19th May 2026 and 23rd May 2026. Testing was performed remotely.

The objective of the re-test was to assess the effectiveness of Nitin's efforts to remediate the issues identified during the original penetration test.

The following domains were considered within the scope of this assessment.

Domain

The scope for this security assessment included, but was not limited to, the following tests:

- Identification of running services
- Vulnerability Assessment
- Penetration Testing of Web Application
- Identification of vulnerable or outdated components and software in use

4.1 TOOLS USED

The following open-source tools were used during this vulnerability assessment exercise:

- Nmap
- SSLScan
- Burp Professional Pro
- SQLMap
- Postman
- Caido

4.2 ASSESSMENT LIMITATION

The ever-changing technology landscape and the increasing sophistication of attacks against web applications are reasons no entity can truthfully claim to identify all security issues nor guarantee the lifetime security of an organization’s web

applications. Note that this point-in-time assessment was based on a best-effort basis. It was also performed only in the environment provided by Nitin. Thus, environmental changes may impact the applicability of the results provided herein.

BlueAlly cannot guarantee 100% coverage for any security assessment.

5 FINDINGS AND RECOMMENDATIONS

5.1 RISK CLASSIFICATION

The remainder of this report describes the vulnerabilities that BlueAlly identified as part of the assessment, their impact, and recommendations for resolving the vulnerabilities. To help determine the risk posed by these vulnerabilities to the Nitin web applications, BlueAlly leveraged the ASVS L1 Security & NIST v3.0 Risk Rating Methodology. The observations were categorized based on technical impact and likelihood, explained below. These impact and likelihood scores may be further modified by Nitin based on the business criticality of the target.

5.2 MEASUREMENT OF IMPACT

The impact is an estimation of the damage potential of a successful exploit of a vulnerability. We'll use the following factors to help qualitatively determine the impact of a vulnerability.

Impact Factor	Description
Classification of Data Loss	If an adversary exploited this vulnerability, what type of data would he/she have stolen? Non-sensitive, non-customer-specific data would reduce the impact of an exploit.
Data Integrity	If an adversary exploited this vulnerability, would he/she have the ability to edit/delete data that they shouldn't have access to? If so, what type of data would they be able to corrupt? Not being able to edit/delete data or the corruption of non-sensitive data would reduce the impact of an exploit.
Blast Radius	How many potential victims would there be if the vulnerability was exploited? For instance, an obscure service with few users would have a smaller victim pool than a major service that is publicly accessible with millions of users. The impact of a successful exploit is reduced with fewer

Impact Factor	Description
	potential victims.

There is no science to determine the impact of using the above factors. If most/all of the factors are pointing to a lower impact, we can evaluate it as Low. If the factors pointing to lower impact are about 50%, then the impact would be rated Medium. Lastly, if most/all of the factors are pointing to a higher impact, we can evaluate it as High.

5.3 Measurement of Likelihood

Likelihood is a qualitative estimation of the probability of an adversary exploiting the vulnerability in question. To determine the likelihood of an exploit, we can consider the following factors:

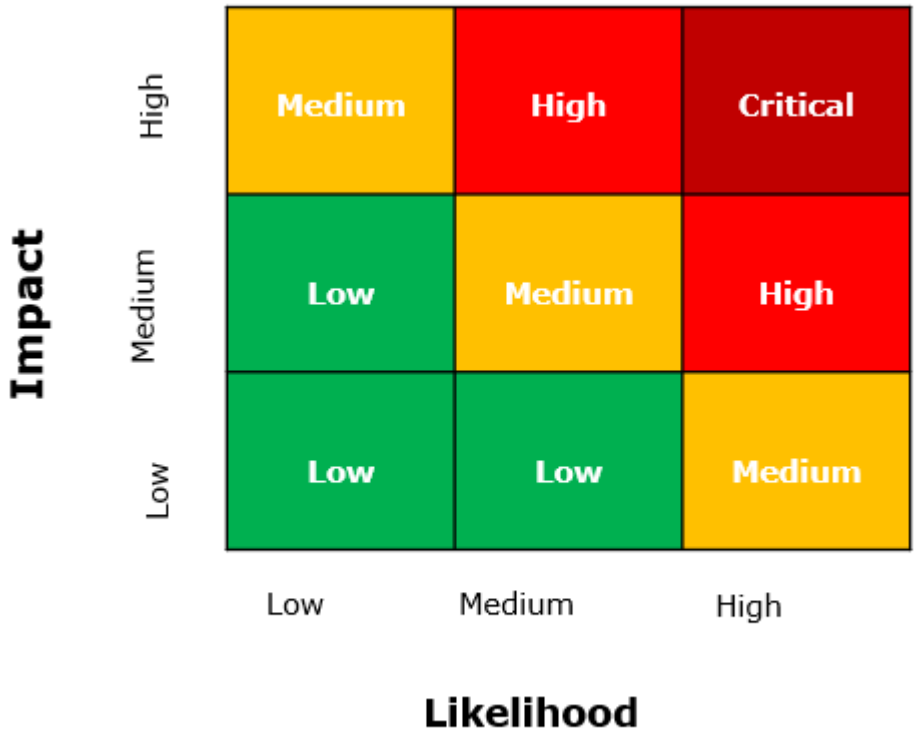
Likelihood Factor	Description
Required Skill Level of Adversary	How technically skilled would an adversary have to be to exploit this vulnerability? • Higher levels of technical skills required would reduce the likelihood of a successful exploit.
Number of Potential Adversaries	How many potential adversaries exist? Technically, any person who can access the system and discover the vulnerability is a potential adversary. • If a smaller number of users have access to the system, the likelihood of a successful exploit is reduced.
Cost of Exploit	How expensive are the systems and resources needed by an adversary to exploit this vulnerability? • More expensive systems and resources required would reduce the likelihood of a successful exploit.
Ease of Exploit	Once the vulnerability has been discovered, how simple is it for an adversary to exploit it? Does it depend on other events? Does it require a specific delivery mechanism?

Likelihood Factor	Description
	• Theoretical vulnerabilities with complex exploit scenarios would reduce the likelihood of the vulnerability.

There is no science to determine the likelihood using the above factors. If most/all of the factors are pointing to a lower likelihood, we can evaluate it as Low. If the factors pointing to lower likelihood are about 50%, then the likelihood would be rated Medium. Lastly, if most/all of the factors are pointing to a higher likelihood, we can evaluate it as High.

5.4 Overall Risk

The following graph illustrates how Impact x Likelihood scores translate to overall Low, Medium, and High-risk ratings:



5.5 ZERO-RISK ISSUES

There are cases where a particular issue doesn't have a direct security impact or likelihood of exploitation. These could be issues related to documentation or semantics that are suggested based on the security assessors' technical and industry experience. Such issues will be reported with an Informational risk rating.

6 VULNERABILITIES

Below is a summary of the vulnerabilities discovered during the assessment, ranked in order of risk:

6.1 WEB APPLICATION FINDINGS

Vulnerability	Risk
Broken Access Control Leading to Account Takeover via API Endpoint	**Medium**
Auction Time Restriction Bypass via Client-Side Enforcement	**Critical**

7 DETAILED VULNERABILITIES

7.1. BROKEN ACCESS CONTROL LEADING TO ACCOUNT TAKEOVER VIA API ENDPOINT

Risk: Medium

Description:

The application implements role-based access control to restrict access to sensitive user information. While the normal application interface prevents lower-privileged users from viewing other users' details, the backend API endpoints responsible for retrieving user account information do not properly enforce authorization checks. A low-privileged authenticated user can directly access these API endpoints to retrieve sensitive information belonging to other users without authorization. The exposed response includes complete user account details, including usernames and passwords stored in clear text. This direct disclosure of credentials enables an attacker to authenticate as any user within the application, leading to full account compromise.

Affected URL: https://www.dev.blackrockgalleries.com/auction-admin/add_admin.php

Steps to reproduce:

Step 1: Log in to the application as a low-privileged user (e.g., 'manager' role).

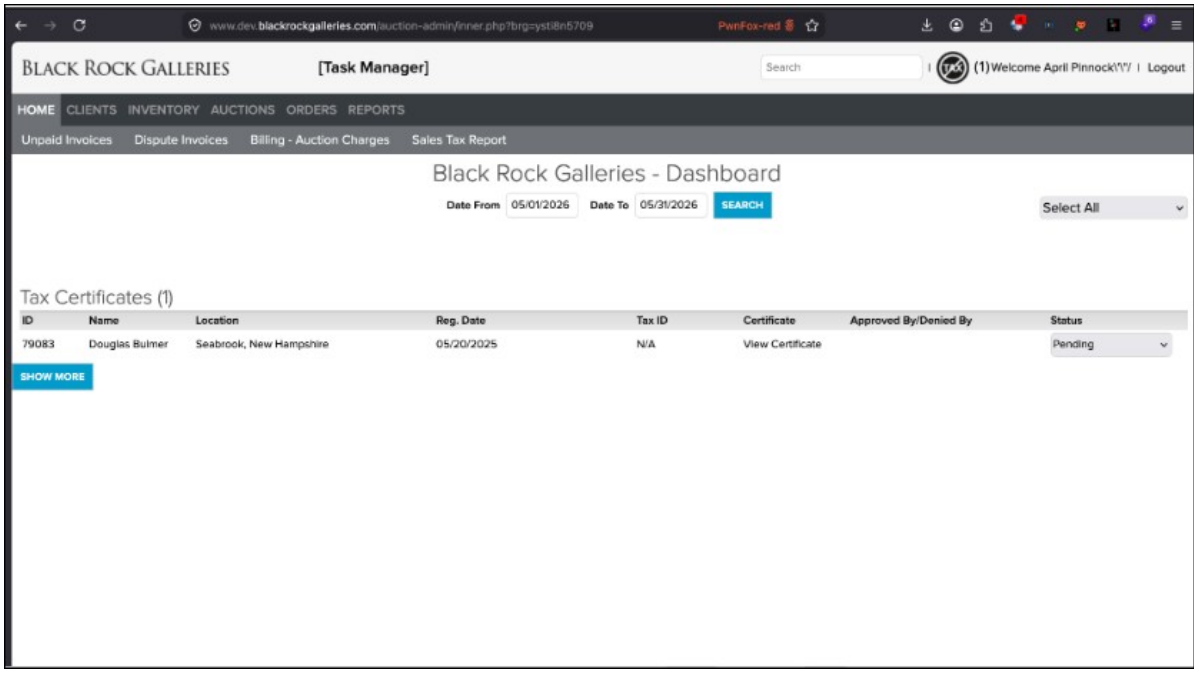


Fig 1: Lower user logged in into admin portal.

Step 2: Construct a direct request to the vulnerable API endpoint, for example: `https://www.dev.blackrockgalleries.com/auction-admin/add_admin.php?id=137` (where 'id' is a valid user ID).

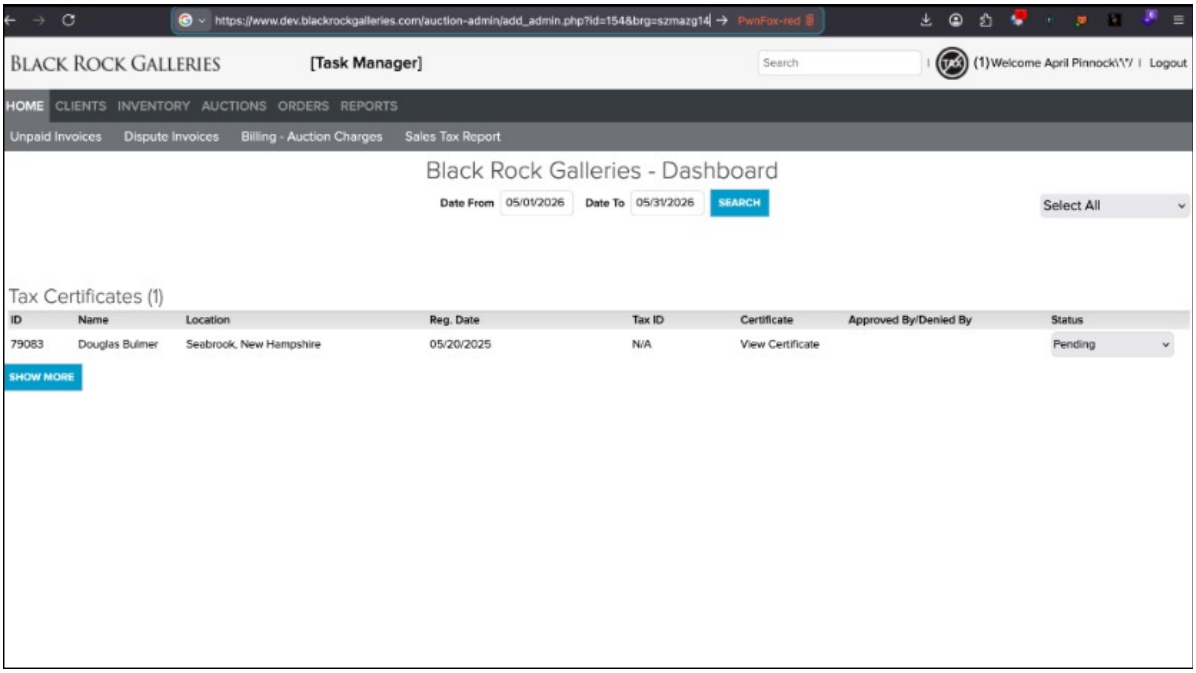


Fig 2: Forced browsing to view admin user details.

Step 3: Observe the response, which contains the complete user account details, including the username and password in clear text.

Step 4: Use the retrieved clear text credentials to log in as the compromised user, confirming account takeover.

Fig 4: Shows clear text password revealed in HTTP response.

Impact and Likelihood:

- **Impact** (High): The impact of this vulnerability is severe, as it allows for complete account takeover of any user within the application, including administrative accounts if present. An attacker can gain unauthorized access to sensitive data, perform actions on behalf of other users, and potentially escalate privileges. This can lead to data breaches, reputational damage, financial losses, and compromise the integrity and confidentiality of the application and its users.
- **Likelihood** (High): Exploitation of this vulnerability is highly likely as it only requires a low-privileged authenticated user to directly access a specific API endpoint. The lack of server-side authorization checks on the API, combined with the clear text disclosure of passwords, makes it straightforward for an attacker to enumerate and compromise user accounts with minimal effort.

Recommendations:

- **Access Control:** Implement robust server-side authorization checks for all API endpoints. Ensure that every request to an API endpoint verifies the user's permissions and role before granting access to sensitive data or functionality. This should be enforced at the backend, independent of any client-side controls.
- **Secure Credential Storage:** Never store passwords in clear text. All user passwords must be stored using strong, one-way cryptographic hashing functions (e.g., bcrypt, Argon2, scrypt) with a unique salt for each password. This prevents the direct disclosure of credentials even if the database is compromised.
- **Principle of Least Privilege:** Ensure that users and roles are granted only the minimum necessary permissions to perform their intended functions. Restrict access to administrative functions and sensitive data to only authorized personnel.
- **Input Validation:** Implement strict input validation for all parameters, especially those used to identify resources (e.g., user IDs). This can help prevent Insecure Direct Object Reference (IDOR) vulnerabilities.

Reference:

https://owasp.org/www-project-top-ten/2021/A01_2021-Broken_Access_Control•

https://cwe.mitre.org/data/definitions/284.html•

https://cwe.mitre.org/data/definitions/256.html•

Back to Summary

7.2. Auction Time Restriction Bypass via Client-Side Enforcement

Risk: Critical

Description:

The auction system implements time restrictions solely at the client-side user interface level. When an auction's designated closing time is reached, the 'Bid' button in the UI becomes disabled, preventing users from submitting bids through standard interaction. However, the backend API endpoint responsible for processing bids does not perform server-side validation to confirm whether the auction is still active or has already concluded. This allows an attacker to bypass the client-side control by directly sending requests to the bidding API endpoint even after the auction has officially closed according to the client-side timer.

Affected URL: https://www.dev.blackrockgalleries.com/processingbidAjax.php

Steps to reproduce:

Step 1: Access an active auction on the platform and observe the countdown timer.

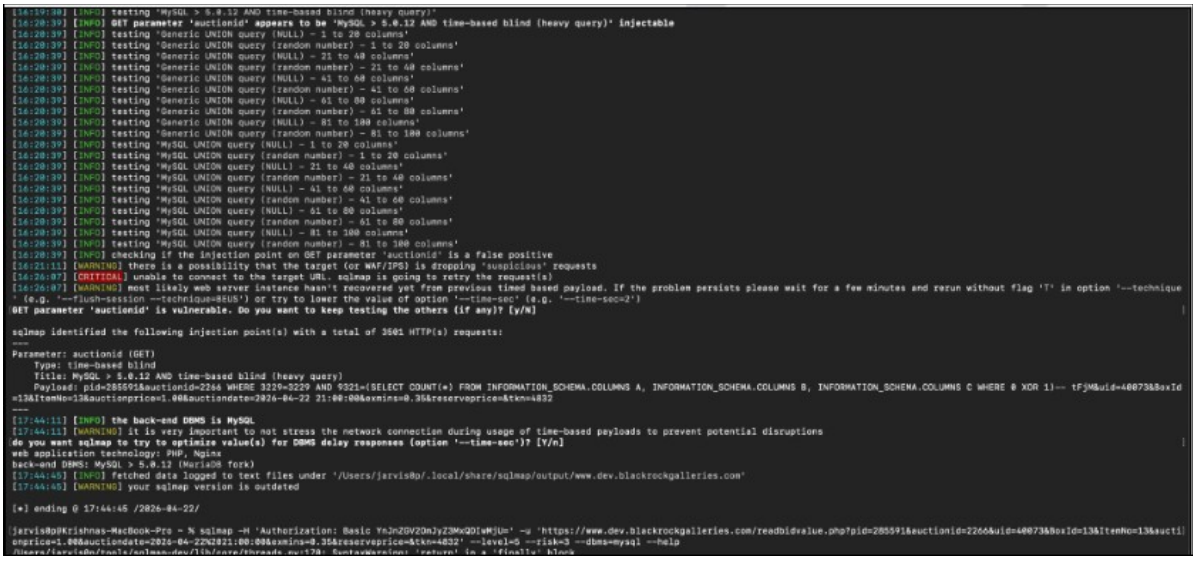


Fig 1: pic1

Step 2: Using a web proxy tool (e.g., Burp Suite), intercept a legitimate bid request made to the 'processingbidAjax.php' endpoint.

Step 3: Wait for the auction timer to expire, at which point the 'Bid' button in the user interface will become disabled.

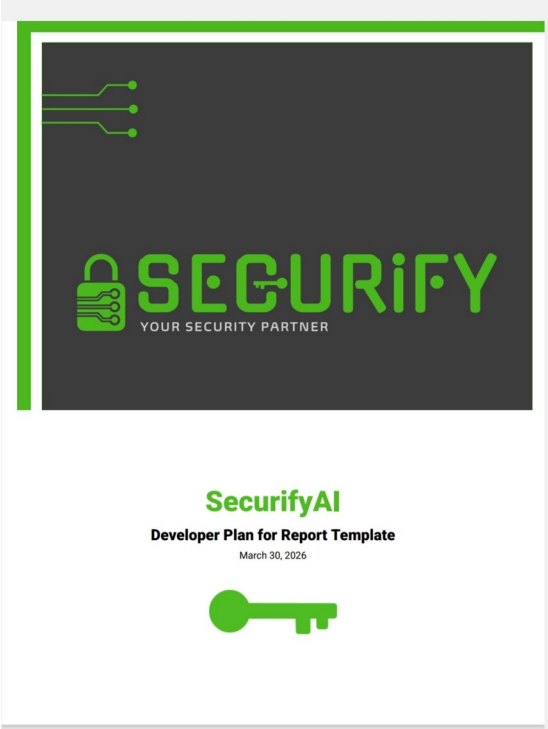


Fig 3: pic3

Step 4: After the auction has closed client-side, resend the intercepted bid request (or a crafted equivalent) directly to the 'processingbidAjax.php' endpoint.

Step 5: Observe that the bid is successfully processed by the server, effectively bypassing the auction's closing time restriction.

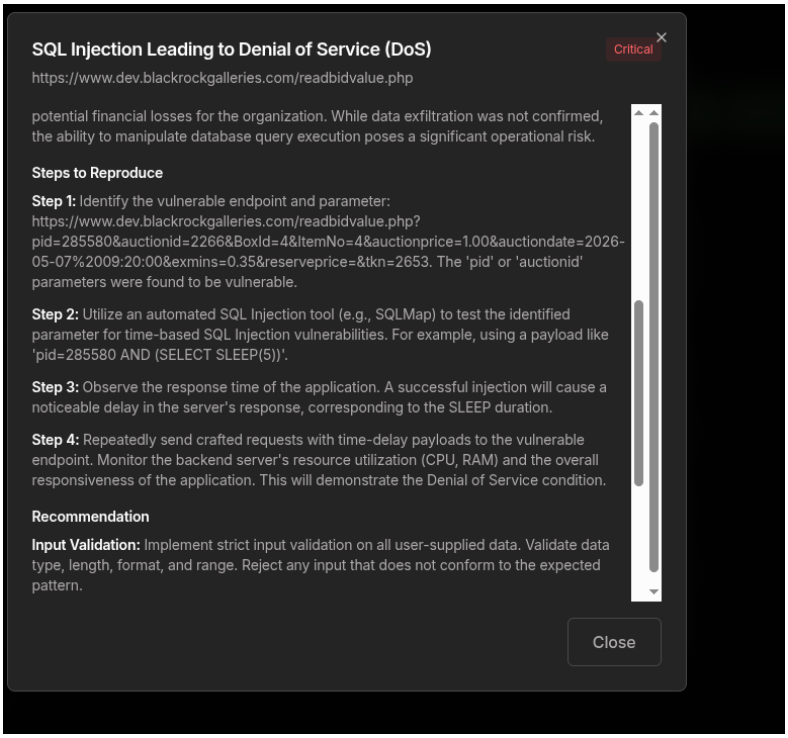


Fig 5: pic5

Impact and Likelihood:

- **Impact** (High): The impact of this vulnerability is significant, as it directly undermines the integrity and fairness of the auction process. Successful exploitation can lead to unauthorized bids being placed after an auction has closed, potentially altering auction outcomes, causing financial losses for legitimate bidders, and damaging the reputation and trustworthiness of the auction platform. This could result in disputes, chargebacks, and a loss of user confidence in the system's fairness and security.
- **Likelihood** (High): Exploitation of this vulnerability is highly feasible. An attacker requires minimal technical skill, primarily the ability to intercept or craft HTTP requests. By observing the legitimate bid request and replaying it after the client-side timer expires, or by directly crafting a request to the API endpoint, the time restriction can be easily circumvented. No complex authentication bypass or advanced techniques are required.

Recommendations:

- **Server-Side Validation:** Implement robust server-side validation for all auction-related actions, especially bid submissions. The server must independently verify the current status of an auction (e.g., open, closed, pending) before processing any bid requests.
- **Business Logic Enforcement:** Ensure that critical business logic, such as auction closing times, is enforced exclusively on the server-side and not solely reliant on client-side

controls. Client-side controls should only be used for user experience enhancements, not for security enforcement.

- **API Security:** All API endpoints handling sensitive operations, like bidding, must include comprehensive checks against the current state of the system and user permissions. Do not trust data or state information provided by the client.
- **Input Validation:** Validate all parameters received by the 'processingbidAjax.php' endpoint against expected values and current auction rules, including the auction's active status.

Reference:

https://cwe.mitre.org/data/definitions/602.html•

https://owasp.org/www-project-top-ten/2021/A04_2021-Insecure_Design•

Back to Summary

8. Appendix A: Penetration Test Scope

V2	Authentication Verification Requirements
2.5.3	Verify that password credential recovery does not reveal the current password in any way
2.5.4	Verify shared or default accounts are not present (e.g., "root", "admin", or "sa").

V4	Access Control Verification Requirements
4.2	Operation Level Access Control
4.2.2	Verify that the application or framework enforces a strong anti-CSRF mechanism to protect authenticated functionality, andthat effective anti-automation or anti-CSRF protection protects unauthenticated functionality
4.3	Other Access Control Considerations
4.3.2	Verify that directory browsing is disabled unless deliberately desired. Additionally, applications should not allow discovery or disclosure of file or directory metadata, such as Thumbs.db.DS_Store, .git, or .svn folders

V5	Validation, Sanitization and Encoding Verification Requirements Control Objective
5.1	Input Validation Requirements
5.1.1	Verify that the application has defenses against HTTP parameter pollution attacks, particularly if the application framework makes no distinction about the source of request parameters (GET, POST, cookies, headers, or environment variables)
5.1.2	Verify that frameworks protect against mass parameter assignment attacks, or that the application has countermeasures to protect against unsafe parameter assignment, such as marking fields private or similar
5.1.3	Verify that all input (HTML form fields, REST requests, URL parameters, HTTP headers, cookies, batch files, RSS feeds, etc) is validated using

V5	Validation, Sanitization and Encoding Verification Requirements Control Objective
	positive validation (whitelisting)
5.1.4	Verify that structured data is strongly typed and validated against a defined schema, including allowed characters, lengt,h and pattern (e.g,. credit card numbers or telephone, or validating that two related fields are reasonable, such as checking that suburb and zip/postcode match)
5.1.5	Verify that URL redirects and forwards only allow whitelisted destinations, or show a warning when redirecting to potentially untrusted content
5.2.1	Verify that all untrusted HTML input from WYSIWYG editors or similar is properly sanitized with an HTML sanitizer library or framework feature
5.2.2	Verify that unstructured data is sanitized to enforce safety measures such as allowed characters and length
5.2.3	Verify that the application sanitizes user input before passing to mail systems to protect against SMTP or IMAP injection
5.2.4	Verify that the application avoids the use of eval() or other dynamic code execution features. Where there is no alternative, any user input being included must be sanitized or sandboxed before being executed
5.2.5	Verify that the application protects against template injection attacks by ensuring that any user input being included is sanitized or sandboxed
5.2.6	Verify that the application protects against SSRF attacks by validating or sanitizing untrusted data or HTTP file metadata, such as filenames and URL input fields, using whitelisting of protocols, domains, paths, and ports
5.2.7	Verify that the application sanitizes, disables, or sandboxes user-supplied SVG scriptable content, especially as they relate to XSS resulting from inline scripts, and foreignObject
5.2.8	Verify that the application sanitizes, disables, or sandboxes user-supplied scriptable or expression template language content, such as Markdown, CSS or XSL stylesheets, BBCode, or similar

V5	Validation, Sanitization and Encoding Verification Requirements Control Objective
5.3	Output encoding and Injection Prevention Requirements
5.3.1	Verify that the output encoding is relevant for the interpreter and context required. For example, use encoders specifically for HTML values, HTML attributes, JavaScript, URL Parameters, HTTP headers, SMTP, and others as the context requires, especially from untrusted inputs (e.g., names with Unicode or apostrophes, such as ねこ or O'Hara).
5.3.2	Verify that output encoding preserves the user's chosen character set and locale, such that any Unicode character point is valid and safely handled
5.3.3	Verify that context-aware, preferably automated - or at worst, manual - output escaping protects against reflected, stored, and DOM-based XSS
5.3.4	Verify that data selection or database queries (e.g., SQL, HQL, ORM, NoSQL) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from database injection attacks
5.3.5	Verify that, where parameterized or safer mechanisms are not present, context-specific output encoding is used to protect against injection attacks, such as the use of SQL escaping to protect against SQL injection
5.3.6	Verify that the application projects against JavaScript or JSON injection attacks, including for eval attacks, remote JavaScript includes, CSP bypasses, DOM XSS, and JavaScript expression evaluation.
5.3.7	Verify that the application protects against LDAP Injection vulnerabilities, or that specific security controls to prevent LDAP Injection have been implemented
5.3.8	Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding
5.3.9	Verify that the application protects against Local File Inclusion (LFI) or Remote File Inclusion (RFI) attacks
5.3.10	Verify that the application protects against XPath injection or XML

V5	Validation, Sanitization and Encoding Verification Requirements Control Objective
	injection attacks
5.5	Deserialization Prevention Requirements
5.5.1	Verify that serialized objects use integrity checks or are encrypted to prevent hostile object creation or data tampering
5.5.2	Verify that the application correctly restricts XML parsers to only use the most restrictive configuration possible and to ensure that unsafe features, such as resolving external entities, are disabled to prevent XXE
5.5.3	Verify that deserialization of untrusted data is avoided or is protected in both custom code and third-party libraries (such as JSON, XML and YAML parsers)
5.5.4	Verify that when parsing JSON in browsers or JavaScript-based backends, JSON.parse is used to parse the JSON document. Do not use eval() to parse JSON

V7	Error Handling and Logging Verification Requirements
7.4	Error Handling
7.4.1	Verify that a generic message is shown when an unexpected or security-sensitive error occurs, potentially with a unique ID that support personnel can use to investigate

V8	Data Protection Verification Requirements
8.3	Sensitive Private Data
8.3.1	Verify that sensitive data is sent to the server in the HTTP message body or headers, and that query string parameters from any HTTP verb do not contain sensitive data

V9	Communications Verification Requirements
9.1	Communications Security Requirements

V9	Communications Verification Requirements
9.1.1	Verify that secured TLS is used for all client connectivity, and does not fall back to insecure or unencrypted protocols.
9.1.2	Verify using online or up-to-date TLS testing tools that only strong algorithms, ciphers, and protocols are enabled, with the strongest algorithms and ciphers set as preferred
9.1.3	Verify that old versions of SSL and TLS protocols, algorithms, ciphers, and configurations are disabled, such as SSLv2, SSLv3, or TLS 1.0 and TLS 1.1. The latest version of TLS should be the preferred cipher suite

V11	Business Logic Verification Requirements
11.1.1	Verify that the application will only process business logic flows for the same user in sequential step order and without skipping steps
11.1.2	Verify the application will only process business logic flows with all steps being processed in realistic human time, i.e., transactions are not submitted too quickly
11.1.3	Verify the application has appropriate limits for specific business actions or transactions, which are correctly enforced on a per-user basis
11.1.4	Verify the application has sufficient anti-automation controls to detect and protect against data exfiltration, excessive business logic requests, excessive file uploads or denial of service attacks
11.1.5	Verify the application has business logic limits or validation to protect against likely business risks or threats, identified using threat modelling or similar methodologies

V12	File and Resources Verification Requirements
12.1	File Upload Requirements
12.1.1	Verify that the application will not accept large files that could fill up storage or cause a denial of service attack
12.2	File Integrity Requirements

V12	File and Resources Verification Requirements
12.2.1	Verify that files obtained from untrusted sources are validated to be of the expected type based on the file's content
12.3	File Execution Requirements
12.3.1	Verify that user-submitted filename metadata is not used directly with system or framework file and URL API to protect against path traversal
12.3.2	Verify that user-submitted filename metadata is validated or ignored to prevent the disclosure, creation, updating, or removal of local files (LFI)
12.3.3	Verify that user-submitted filename metadata is validated or ignored to prevent the disclosure or execution of remote files (RFI), which may also lead to SSRF
12.3.4	Verify that the application protects against reflective file download (RFD) by validating or ignoring user-submitted filenames in a JSON, JSONP, or URL parameter, the response Content-Type header should be set to text/plain, and the Content-Disposition header should have a fixed filename
12.3.5	Verify that untrusted file metadata is not used directly with system API or libraries, to protect against OS command injection
12.3.6	Verify that the application does not include and execute functionality from untrusted sources, such as unverified content distribution networks, JavaScript libraries, node npm libraries, or server-side DLLs
12.5	File Download Requirements
12.5.1	Verify that the web tier is configured to serve only files with specific file extensions to prevent unintentional information and source code leakage. For example, backup files (e.g., .bak), temporary working files (e.g., .swp), compressed files (.zip, .tar.gz, etc,) and other extensions commonly used by editors should be blocked unless required
12.5.2	Verify that direct requests to uploaded files will never be executed as HTML/JavaScript content

V13	API and Web Service Verification Requirements
13.1	Generic Web Service Security Verification Requirements
13.1.1	Verify that all application components use the same encodings and parsers to avoid parsing attacks that exploit different URI or file parsing behavior that could be used in SSRF and RFI attacks
13.1.2	Verify that access to administration and management functions is limited to authorized administrators
13.1.3	Verify API URLs do not expose sensitive information, such as the API key, session tokens, etc
13.1.4	Verify that authorization decisions are made at both the URI, enforced by programmatic or declarative security at the controller or router, and at the resource level, enforced by model-based permissions
13.1.5	Verify that requests containing unexpected or missing content types are rejected with appropriate headers (HTTP response status 406 Unacceptable or 415 Unsupported Media Type)
13.2	RESTful Web Service Verification Requirements
13.2.1	Verify that enabled RESTful HTTP methods are a valid choice for the user or action, such as preventing normal usersfrom using DELETE or PUT on protected API or resources
13.2.2	Verify that JSON schema validation is in place and verified before accepting input
13.2.3	Verify that RESTful web services that utilize cookies are protected from Cross-Site Request Forgery via the use of at least one or more of the following: triple or double submit cookie pattern (see references), CSRF nonces, or ORIGIN request header checks
13.2.4	Verify that REST services have anti-automation controls to protect against excessive calls, especially if the API is unauthenticated
13.2.5	Verify that REST services explicitly check the incoming Content-Type to be the expected one, such as application/XML or application/JSON
13.2.6	Verify that the message headers and payload are trustworthy and not modified in transit. Requiring strong encryption for transport (TLS only) may be sufficient in many cases as it provides both

V13	API and Web Service Verification Requirements
	confidentiality and integrity protection. Permessage digital signatures can provide additional assurance on top of the transport protections for high-security applications but bring with them additional complexity and risks to weigh against the benefits
13.3	SOAP Web Service Verification Requirements
13.3.1	Verify that XSD schema validation takes place to ensure a properly formed XML document, followed by validation of each input field before any processing of that data takes place
13.3.2	Verify that the message payload is signed using WS-Security to ensure reliable transport between client and service
13.4	GraphQL and other Web Service Data Layer Security Requirements
13.4.1	Verify that query whitelisting or a combination of depth limiting and amount limiting should be used to prevent GraphQL or data layer expression denial of service (DoS) as a result of expensive, nested queries. For more advanced scenarios, query cost analysis should be used
13.4.2	Verify that GraphQL or other data layer authorization logic should be implemented at the business logic layer instead of the GraphQL layer

V14	Configuration Verification Requirements
14.2	Dependency
14.2.1	Verify that all components are up to date
14.2.2	Verify that all unneeded features, documentation, samples, configurations are removed, such as sample applications, platform documentation, and default or example users
14.2.3	Verify that if application assets, such as JavaScript libraries, CSS stylesheets or web fonts, are hosted externally on a content delivery network (CDN) or external provider, Subresource Integrity (SRI) is used to validate the integrity of the asset
14.3	Unintended Security Disclosure Requirements

V14	Configuration Verification Requirements
14.3.1	Verify that web or application server and framework error messages are configured to deliver user actionable, customized responses to eliminate any unintended security disclosures
14.3.2	Verify that web or application server and application framework debug modes are disabled in production to eliminate debug features, developer consoles, and unintended security disclosures
14.3.3	Verify that the HTTP headers or any part of the HTTP response do not expose detailed version information of system components
14.4	HTTP Security Headers Requirements
14.4.1	Verify that every HTTP response contains a content type header specifying a safe character set (e.g., UTF-8, ISO 8859-1)
14.4.2	Verify that all API responses contain Content-Disposition: attachment; filename="api.json" (or other appropriate filename for the content type).
14.4.3	Verify that a content security policy (CSPv2) is in place that helps mitigate impact for XSS attacks like HTML, DOM, JSON, and JavaScript injection vulnerabilities
14.4.4	Verify that all responses contain X-Content-Type-Options: nosniff
14.4.5	Verify that HTTP Strict Transport Security headers are included on all responses and for all subdomains, such as Strict-Transport-Security: max-age=15724800; includeSubdomains
14.4.6	Verify that a suitable "Referrer-Policy" header is included, such as "no-referrer" or "same-origin
14.4.7	Verify that a suitable X-Frame-Options or Content-Security-Policy: frameancestors header is in use for sites where content should not be embedded in a third-party site
14.5	Validate HTTP Request Header Requirements
14.5.1	Verify that the application server only accepts the HTTP methods in use by the application or API, including pre-flight OPTIONS
14.5.2	Verify that the supplied Origin header is not used for authentication or

V14	Configuration Verification Requirements
	access control decisions, as the Origin header can easily be changed by an attacker
14.5.3	Verify that the cross-domain resource sharing (CORS) Access-Control-AllowOrigin header uses a strict white-list of trusted domains to match against and does not support the "null" origin
14.5.4	Verify that HTTP headers added by a trusted proxy or SSO devices, such as a bearer token, are authenticated by the application